

Reviewer Pack — Plünnecke-Ruzsa Doubling of Constraint Vectors Lower-Bounds ...

Entry 6ae7590ec4aa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 09:45:23 UTC

Plünnecke-Ruzsa Doubling of Constraint Vectors Lower-Bounds SoS Degree for 3-XOR

Entry ID: 6ae7590ec4aa **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 09:45:23 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics in F_2^n : the Plünnecke-Ruzsa doubling constant $K(A)=|A+A|/|A|$ and its log version $\sigma(A)=\log_2|A+A|-\log_2|A|$ of a finite set $A \subseteq F_2^n$ (Freiman-Ruzsa / Bogolyubov / Sanders' polynomial Bogolyubov tradition). The mathematical branch is harmonic / arithmetic combinatorics on abelian groups, distinct from polyhedral / Newton-polytope geometry, and rarely deployed in SoS lower bounds for CSPs. **Field B** (complexity object): Sum-of-Squares refutation degree $d_{\text{SoS}}(F)$ of unsatisfiable 3-XOR instances F over $GF(2)$, accessed via the Grigoriev-Schoenebeck combinatorial proxy $g^*(F) = \min\{|T| : \bigoplus_{i \in T} a_i = 0 \in F_2^n \text{ and } \bigoplus_{i \in T} b_i = 1 \in F_2\}$, which sandwiches SoS degree to within ± 1 by the Schoenebeck (2008) characterization for 3-XOR.

Statement:

For every unsatisfiable 3-XOR instance $F = ((a_i, b_i))_{i=1..m}$ on $n \leq 20$ variables with constraint multiset $A = \{a_1, \dots, a_m\} \subseteq F_2^n$, the Schoenebeck combinatorial SoS-degree satisfies $g(F) \geq \sigma(A) + 1$, where $\sigma(A) = \log_2|A+A| - \log_2|\text{set}(A)|$. Equivalently, $|A+A| \leq |\text{set}(A)| \cdot 2^{\{g(F)-1\}}$. A single (F, A) violating this inequality refutes the conjecture.

Rationale (proposer's reasoning):

Schoenebeck's theorem says small SoS degree forces a short additive dependency among constraint vectors; Plünnecke-Ruzsa says short additive dependencies push A toward an affine-subspace structure with low doubling. The bridge predicts a monotone, never-before-quantified link between structurelessness (large doubling) of the constraint vectors and the unavoidable length of a Schoenebeck certificate, transferring Freiman-Ruzsa intuition into proof complexity.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8f4a584b1b1a5787

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 5 (n in $\{8,10,12,14,16\}$) $\times$ 3 (α in $\{1.2,2.0,3.0\}$) $\times$ 200 = 3000 random unsat 3-XOR instances, the inequality $g(F) \geq \sigma(A)+1$ must hold on $\geq 99\%$ of instances ($\geq 2970/3000$), with the per- n mean gap $E[g^*(F)-\sigma(A)]$ non-decreasing across $n=8 \rightarrow 16$. Any single instance with $g(F) < \sigma(A)+1$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Plunnecke-Ruzsa doubling sum-of-squares 3-XOR Grigoriev Schoenebeck - additive combinatorics F_2^n Freiman-Ruzsa SoS degree lower bound CSP refutation - sumset doubling constant 3-XOR sum-of-squares hierarchy Schoenebeck degree

Top relevant hits considered: - [<http://arxiv.org/abs/2507.01005v1>] Discovery and study of the eclipsing variable star Grigoriev 1 - [<http://arxiv.org/abs/cond-mat/0701009v2>] Electron mobility on a surface of dielectric media: influence of surface level atoms - [<http://arxiv.org/abs/1812.10216v2>] Robust Approach for Rotor Mapping in Cardiac Tissue - [<http://arxiv.org/abs/1701.04521v1>] Sum of squares lower bounds for refuting any CSP - [<http://arxiv.org/abs/quant-ph/0305179v3>] Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range - [<http://arxiv.org/abs/1508.01127v1>] Encoding CSP into CCS (Extended Version) - [<http://arxiv.org/abs/1709.07966v7>] High Degree Sum of Squares Proofs, Bienstock-Zuckerberg hierarchy and Chvatal-Gomory cuts - [<http://arxiv.org/abs/1710.06598v2>] A Bounded Degree Lasserre Hierarchy with SOCP Relaxations for Global Polynomial Optimization and Applications

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import json
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor(x, y):
        return x ^ y

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
```

```

    for i in range(m):
        if A[i][i] == 0:
            for j in range(i + 1, m):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                continue
        pivot = A[i][i]
        for j in range(n):
            A[i][j] = A[i][j] // pivot
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    return A

def min_weight_inconsistent_subsets(A, b):
    m, n = len(A), len(A[0])
    A_b = [row + b for row in A]
    g_star = 0
    for weight in range(1, m + 1):
        for subset in combinations(range(m), weight):
            if all(all(A_b[i][j] == 0 for j in range(n)) for i in subset):
                g_star = max(g_star, weight)
    return g_star

def sigma(A):
    A_plus_A = set()
    for a, b in product(A, repeat=2):
        A_plus_A.add(xor(a, b))
    return len(set(A)), len(A_plus_A), len(A_plus_A) / len(A)

n = random.choice([8, 10, 12, 14, 16])
alpha = random.choice([1.2, 2.0, 3.0])
m = int(n * alpha)

F = []
for _ in range(m):
    clause = [random.randint(0, 1) for _ in range(n)]
    b = random.randint(0, 1)
    F.append((clause, b))

A = [F[i][0] for i in range(len(F))]
set_A_size, A_plus_A_size, sigma_value = sigma(A)
g_star = min_weight_inconsistent_subsets(A, [F[i][1] for i in range(len(F))])

metric_name = "g*(F) -  $\sigma(A)$ "
metric_value = g_star - sigma_value
instances_tested = 1
conjecture_holds = g_star >= sigma_value + 1
counterexample = "" if conjecture_holds else f"Instance with n={n}, alpha={alpha}"

return {
    "metric_name": metric_name,

```

```

        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_16090c09.py", line 92, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_16090c09.py", line 70, in run_trial
    set_A_size, A_plus_A_size, sigma_value = sigma(A)
                                              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_16090c09.py", line 56, in sigma
    A_plus_A.add(xor(a, b))
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_16090c09.py", line 21, in xor
    return x ^ y
           ~^~
TypeError: unsupported operand type(s) for ^: 'list' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `xor()` helper (xoring two lists), so no data was produced and the pre-registered support condition could not be evaluated. | next: Fix the `xor()` helper to operate on integer bitmask representations of F_2^n vectors (or convert lists via `tuple/int`) and re-run the 3000-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	25848
2	propose	claude_max	opus	0	0	140226
3	preregistration	claude_max	opus	0	0	7314
4	novelty	claude_max	opus	0	0	3321
5	novelty	claude_max	opus	0	0	6933
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17214
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11086
8	judge	claude_max	opus	0	0	8046

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 219988 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6ae7590ec4aa.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ae7590ec4aa.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6ae7590ec4aa.tar.gz` (if generated)